

PYTHON

Module 25: Iterator Protocol & Traversal

CODING

Understanding Python Iterators

An iterator is an object that contains a countable number of values and allows you to traverse through the data one item at a time. At CodeHive, we look at iterators as the engine behind Python's looping mechanisms.

- ****The Iterator Protocol****: Technically, an iterator is any object that implements two specific dunder methods: `__iter__()` and `__next__()`.

Iterable vs. Iterator

It is important to distinguish between a container and the object that traverses it.

- **Iterable**: Objects like Lists, Tuples, Dictionaries, and Strings are *iterable*. They contain data but are not iterators themselves.

- **The iter() Method**: You can obtain an iterator from any iterable using the `iter()` function.

- **The next() Method**: Once you have an iterator, you use the `next()` function to retrieve the subsequent value in the sequence.

Automated Iteration: for Loops

When you use a `for` loop in Python, the language handles the iterator protocol for you behind the scenes.

1. It calls `iter()` on the object to create an iterator.
2. It repeatedly calls `next()` to get values.
3. It automatically catches the `StopIteration` error to end the loop gracefully.

Building Custom Iterators

You can turn any Python class into an iterator by defining its internal logic:

- `**__iter__()`: Initializes the state and must return the iterator object itself (`return self`).

- `**__next__()`: Contains the logic to compute and return the next item. This is where the 'counting' or 'traversing' logic lives.

Infinite Sequences & StopIteration

Without a boundary, an iterator can generate values forever. To prevent this, Python uses the `StopIteration` exception.

- ****Logic****: Inside your `__next__()` method, add a condition. If the condition is met (e.g., reaching a limit), use `raise StopIteration` to signal that no more values are available.
-

****CodeHive Pro-Tip****: This is how Python creates efficient memory-saving sequences that compute values on-demand rather than storing them all at once.